INDEX

NAME S. Shashank SUBJECT

STD. DIV. ROLL NO. SCHOOL

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
		Finding Time Complexity of Algorithm		
1)	6/8/26	Problem 1		R
2)	6/8/26	Problem 2		R
3)	6/8/26	Problem 3		R
4)	6/8/26	Problem 4		R
5)	6/8/26	Problem 5		R
		Divide And Conquer		
1)	5/9/26	Number of zeroes in a given array		R
2)	5/9/26	Majority Element		R
3)	5/9/26	Finding Floor Value		R
4)	5/9/26	Two elements sum to x		R
5)	5/9/26	Implementation of quick sort		R
		Greedy Algorithms		
1)	31/8/26	Gr-Coin Problem		R
2)	31/8/26	Gr-Lectures Problem		R
3)	31/8/26	Gr-Burger Problem		R
4)	31/8/26	Gr-Sum Max Problem		R
5)	31/8/26	Gr-Product of Array Elements minimum		R
		Dynamic Programming		
1)	2/9/26	Playing with numbers		R
2)	2/9/26	Playing with Chessboard		R
3)	2/9/26	Longest Common Subsequence		R
4)	2/9/26	Longest non-decreasing Subsequence		R

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
		Competitive Programming		
1)	10/10/26	Finding duplicate $O(n^2)$ Time Complexity $O(1)$ Time Complexity.		✓
2)	10/10/26	Finding duplicate $O(n)$ Time complexity $O(1)$ Space Complexity.		✓
3)	10/10/26	Print intersection of 2 Sorted arrays - $O(m*n)$ Time Complexity $O(1)$ space complexity.		✓
4)	11/10/26	Intersection of 2 Sorted arrays $O(m+n)$ time complexity $O(1)$ Space Complexity.		✓
5)	10/10/26	Pair with difference $O(n^2)$ time completity $O(1)$ Space complexity.		✓
6)	10/10/26	Pair with difference $O(n)$. Time complexity $O(1)$ Space Complexity.		✓

Completed ✓

28/8/26

Finding Time Complexity of the Given Algorithm

Aim:

To find the counter variable and time complexity.

Algorithm:

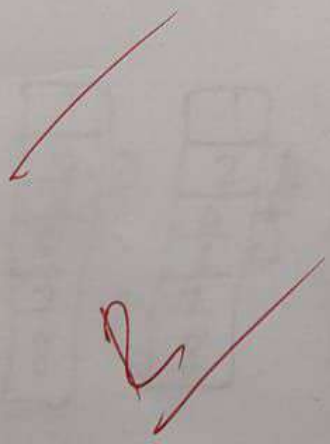
```
void function(int n)
{
   int c=0;
   int s=1;
   while (s<=n)
   {
      i++;
      s+=i;
   }
}
```

Program:

```
#include <stdio.h>
int main()
{
   int n, count=0;
   scanf("%d", &n);
   int s=1; count++;
   int i=1; count++;
   while(s<=n, count++)
   {
      i++; count++
      s+=i; count++
```

Output:

Input	Expected	Got
9	12	12
4	9	9



Result:

The program was written and executed successfully.

6/8/26

Finding Complexity using Counter Method

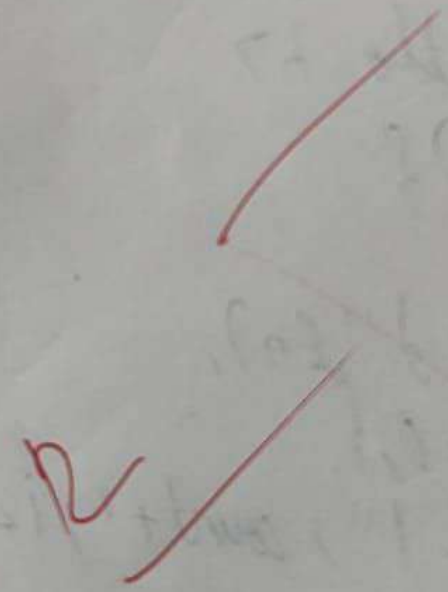
Aim:

Convert the following algorithm into Program and find its time complexity using counter method.

Algorithm:

```
void func (int n)
{
  if(n==1)
    {printf("*");
    }
else
  {
    for(int i=0;i<=n;i++)
      {for(int j=1;j<=n;j++)
        {printf("*");
        printf("*");
        break
        }
      }
  }
}
```

Input	Output
12	31
25 1000	54 5002



```
else {
    count++;
    for (int i=1; count++; i<=n; count++; i++) {
        for (int j=1; count++; j<=n; count++; j++) {
            count++;
            count++;
            break;
        }}}
    Printf ("count");
}
```

Result:

The program has been executed succes

Program:

```
#include <stdio.h>
int main()
{
  int count = 0;
  int n;
  scanf("%d", &n);
  if (n == 1)
    {
      count++
      printf("*");
      count++;
    }
  else
  {
  count++;
  for (int i = 1; count++; i<=n; count++, i++)
     {
     for (int j = 1; count++; j<=n; count++; j++)
       counter = count + 2;
       break;
}}}
```

Result:

The program has been written and executed successfully.

4)

Finding Time Complexity of an Algorithm

Aim:

To find the time complexity of an algorithm using counter Method

Algorithm:

```
void function(int n)
{
 int c=0;
 for(int i=n/2 ; i<n; i++)
  {
  for(int j =1; j<n; j =2*j)
   {
   for(int k=1; k<n; k=k*2)
     c++
   }
  }
```

Program:

```
# include <stdio.h>
int main()
{
 int n;
 scanf("%d", &n);
 int count=1;
```

```
int i, j, k;
for (i=n/2 ; i<n; i++)
 {
  count ++ ;
  for (j= 1; j<n; j = 2*j)
   {
   count++;
   for (k=1; k<n; k<= k* 2)
    {
     count++;
     count++;
    }
    count ++;
  }
  count++;
}
count++;
printf("%d", count);
return 0;
}
```

Result:

The algorithm has been executed successfully.

Input	Output
12	11
1234	19

Input	Output
4	3 0
10	212

```
    c = c+4;
    n = n/10;
  }
printf("%d", c);
}
```

Result:

The program has been executed successfully

2) Finding Time Complexity of an Algorithm

Aim:

To find the time complexity of an algorithm using counter method.

Algorithm:

```
void rev=0, remainder;
while (n!=0)
{
 remainder = n%10;
 rev = rev *10+ remainder;
 n/=10;
}
print (rev);
}
```

Program:

```
#include <stdio.h>
int main ()
{
 int n;
 scanf ("%d", &n);
 int c = 3;
 while (n!=0)
 {
```

```
int count = 0;
for(int i=0; i<n; i++){
  if(a[i] == 0)
    count++;
}
printf("%d", count);
return 0;
}
```

Result:

The program has been executed successfully

Input	Output
5 1 1 1 0 0	2
9 0 0 0 0 0 0 0 0	8

(2) Divide And Conquer

Number of Zeros in A Given Array.

Aim:

To find the number of zeros in the given array Divide and conquer method.

Algorithm:

```
int a[n];
int count = 0;
for(int i=0;i<n;i++)
  if (a[i] == 0)
    count++;
printf("%d", count);
```

Program:

```
int main()
{
  int n;
  scanf("%d", &n);
  int a[n];
  for(int i=0; i<n; i++){
    scanf("%d", &a[i]);
  }
```

Program :

```
#include<stdio.h>
int majorr(int arr[], int l, int r){
    if(l==r)
        return arr[l];
    int m = (l+r)/2
    int left = majorr(arr, l, m);
    int right = majorr(arr, m+1, r);
    if(left == right) return left;
    int lc=0, rc=0;
    for(int i=l; i<=r; i++){
        if(arr[i]==left) lc++;
        else rc++;
    }
    return (lc>rc)? left:right;
}
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++)
        scanf("%d",&arr[i]);
    printf("%d", majorr(arr,0,n-1));
}
```

Result:

Thus the program has been executed successfully

Input	Output
3 3 2 3	3

Finding Majority Element

Aim: To find the majority elements in a array of size n.

Algorithm:

```
Major(arr, l, r)
if l==r
  return arr[l]
  mid = (l+r)/2
leftmajor = major(arr, l, m)
rightmajor = major(arr, m+1, r)
if leftmajor = rightmajor
    return leftmajor
leftcount = 0
rightcount = 0
for i=0l ; i<n ; i++
  if arr[i] = left major
        leftcount = leftcount++;
  if arr[i] = right major
        rightcount = rightcount++;
if leftcount > rightcount
  return leftcount ;
else
  return rightcount ;
```

Program:

```
#include <stdio.h>
int main() {
   int n, x;
   scanf("%d", &n);
   int arr[n];
   for(int i=0; i<n; i++)
      scanf("%d", &arr[i]);
   int low = 0; high = n-1;
   int floor = -1; scanf("%d", &x);
   while (low <= high) {
      int mid = (low+high)/2;
   if (arr[mid] == x)
      floor = arr[mid];
      break;
   else if (arr[mid] < x)
      floor = arr[mid];
      low = mid-1;
   }
}
```

Result:

The program has been executed successfully.

Input	Output
6 1 2 8 10 12 19 5	2
5 10 22 85 108 129 100	85

Finding Floor Value

Aim:

To write a program find the floor value of a number in an array using divide and conquer.

Algorithm:

```
Floor(A, n, x)
{
 low = 0
 high = n-1
 floor = -1
 while low <= high.
    mid = (low + high)/2
    if A[mid] = x
       floor = A[mid]
       break
    else if A[mid] < x
       floor = A[mid]
       low = mid+1
    else
       high = mid-1
 return floor.
```

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int n, x;
    scanf("%d", &n);
    int arr = malloc(n * sizeof(int));
    for(int i = 0; i < n; i++) scanf("%d", &arr[i]);
    scanf("%d", &x);
    int i = 0, j = n-1;
    while(i < j){
        int sum = arr[i] + arr[j];
        if(sum == x)
            printf("%d\n%d\n", arr[i], arr[j]
            free(arr);
        }
        else if(sum < x
            i++
        else
            i--
    }printf("No");
}
```

Result:

The program has been executed successfully.

Input	Output
4	4
2	1 8
4	
8	
18	
14	No
5	
2	
4	
6	
10	
100	

4)

Two elements sum to x

Aim: To find whether there exists two elements in a sorted array whose sum is equal to a given value x using Divide and Conquer method.

Algorithm:

```
FindTwoSum (A, n, x){
left = 0
right = n - 1
while left < right {
    sum = A[left] + A[right]
    if sum = x
      print ( A[left], A[right]

    else if sum < x
      left = left + 1

    else
      right = right - 1
print ("No")
```

```
    for(int j = low; j < high; j++) {
        if (arr[j] < pivot)
            swap arr[i++] = arr[j]
            arr[
        return i + 1;

void quicksort (int arr[], int low, int high){
    if (low < high)
        int pi = partition(arr, low, high);
        quicksort (arr, low, pi-1);
        quicksort (arr, pi+1, high);
    }
}

int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i=0; i<n; i++)
        scanf("%d", &arr[i]);
        quicksort(arr, 0, n-1);
    for(int i=0; i<n; i++)
        scanf("%d", arr[i]);
    return 0;
}
```

Result:

The program has been executed successfully

Input	Output
5 67 34 12 98 19 10 156 78 90 32 56 11 10 90 114	12 34 07 78 98 7 10 11 32 56 78 90 90 114

5)

Implementation of Quick Sort

Aim:

To sort a list of elements in ascending order using quick sort algorithm.

Algorithm:

```
Quicksort (A, low, high)
 if low < high
   pivot = A[high]
   i = low - 1
   for j = low ; j < high ; j++
      if A[j] < pivot
        i = i + 1
        swap A[i] and A[j]
   p = i + 1
   Quicksort (A, low, p-1)
   Quicksort (A, p+1, high)
```

Program:

```
# include <stdio.h>
void swap (int *a, int *b) {
 int t = *a ; a = *b ; b* = t
}
int partition (int arr[], int low, int high) {
 int pivot = arr[high] ;
```

Input	Output
49	5



1)

Greedy Method
Coin Problem

Aim: To write a problem to find the minimum no. of coins or notes needed using greedy method

Program:

```
# include <stdio.h>
int main()
{
 int v;
 scanf("%d",&v);
 int demo[] = {1000,500,100,50,20,10,5,2,1};
 int count = 0;
 for (int i=0;i<9;i++)
 {
  count += v /demo[i];
  v%. = denominations [i];
 }
 printf("%d", count);
 return 0;
}
```

Result:

The program has been executed successfully.

```
        if(used[j]==0 && a[i][j]==g[i][j])
        {
          count++;
          used[j]=1;
          break;
        }
      }
    }
    printf("%d", count);
    return 0;
}
```

Result:

The program has been executed successfully.

Input	Output
2 1 2 3 1 2 3	2

Cookies Problem

Aim:

To write a problem to maximize the no. of content children and output maximum number using greedy method.

Program:

```
#include <stdio.h>
int main()
{
  int n, m;
  scanf("%d", &n);
  int g[n];
  for(int i=0; i<n; i++)
  {
    scanf("%d", g[i]);
  }
  scanf("%d", &m);
  int s[m];
  for(int i=0; i<m; i++)
  {
    scanf("%d", &s[i]);
  }
  int count=0;
  for(int i=0; i<n; i++)
  {
    for(int j=0; j<m; j++)
    {
```

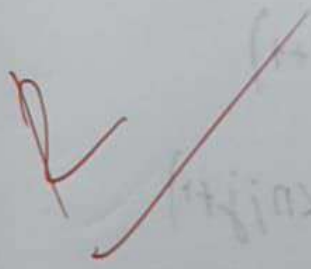
```
int sum = 0;
for(int i=0; i<n; i++)
  {
  sum += (pow (n,i) * arr[i]);
  }
  printf ("%d", sum);
}
```

Result:

The program has been executed successfully.

Input	Output
3 1 3 2	18





Burger Problem

Aim:

To write a program to solve Burger Problem using greedy method.

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
int main()
{
 int n;
 scanf("%d",&n);
 int arr[n];
 for(int i=0;i<n;i++)
   scanf("%d",&arr[i]);
 int temp;
 for(int i=0;i<n;i++)
   {
    for(int j=i+1;j<n;j++)
     {
      if(arr[i]<arr[j])
        {
          temp=arr[i];
          arr[i]=arr[j];
          arr[j]=temp;
        }
     }
   }
```

Input	Output
5 2 5 3 4 0	40



Sum Max Problem

Aim:

To write a program to maximize the sum of given array.

Program:

```
#include<stdio.h>
#include<stdlib.h>
int cmp(const void *a, const void *b);
{
return *(int*)a - *(int*)b
}
int main()
{
  int n;
  scanf("%d", &n);
  int a[n];
  for(int i=0; i<n; i++)
    scanf("%d", &a[i]);
  qsort(a, n, sizeof(int), cmp);
  int sum=0;
  for(int i=0; i<n; i++)
    sum += a[i] * i;
  printf("%d", sum);
}
```

Result:

The program has been executed successfully.

```
for(int i = 0; i < n; i++)
  {
   sum += (a[i] * b[i]);
  }
printf("%d", sum);
return 0;
}
```

Result:

The program has been executed successfully.

Input	Output
3 1 2 3 4 5 6	28



5)

Product of Array Elements - Minimum

Aim:

To write a program to arrange two arrays such that sum of pairwise products are minimised

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
int cmp(const void *a, const void *b)
 {
  return (*(int*)a - *(int*)b);
 }
 int cmp2(const void *a, const void *b)
  {
   return (*(int*) b - *(int *) a);
  }
int main ()
 {
  int n, sum=0;
  scanf("%d", &n);
  int a[n], b[n];
  for(int i=0; i<n; i++)
   {
    qsort(a, n, sizeof(int), cmp1)
    qsort(b, n, sizeof(int), cmp2)
```

```
long dp[n+1];
dp[0] = 1;
for (int i=1; i<=n; i++)
    dp[i] = dp[i-1]
    if (i >= 3)
    dp[i] += dp[i-3]
}
printf("%d", dp[n]);
return 0;
}
```

Result:

Thus the program was successfully written and executed.

Input	Output
6	6
25	8641
100	243828197596721629

Dynamic Programming
Playing with Numbers

Aim:

To find the the total numbers of ways a number can be represented by using 1 or 3.

Algorithm:

Algorithm numbers()
{
Read n
set dp[0] = 1
For i = 1 to n
dp[i] = dp[i-1]
if i >= 3
dp[i] = dp[i] + dp[i-3]
End if
End For
print dp[n]
Stop
}

Program:

```
#include <stdio.h>
int main()
{
 int n;
 scanf("%d", &n);
```

```
for (int i = 1; i<n; i++)
{
  for (int j = 1; j<n; j++)
  {
    if (a[i][j] > a[i][j-1])
      a[i][j] += a[i-1][j];
    else
      a[i][j] += a[i][j-1];
  }
}
printf("%d\n", a[n-1][n-1]);
}
```

Result:

Thus the program to find the highest monetary path was written and executed successfully.

Input	Output
3 1 2 4 2 3 4 8 7 1 3 1 3 1 1 5 1 4 2 1 4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	19 12 28

Aim:
To find the maximum monetary value path from top left cell to bottom right cell while only moving right or down.

Program:

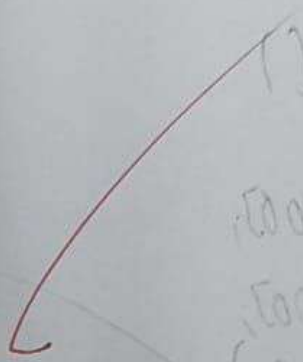
```
#include <stdio.h>
int main()
{
  int n;
  scanf("%d", &n);
  int a[n][n];
  for(int j = 0; j < n; j++)
    {
      scanf("%d", &a[i][j]);
    }
  for(int i = 1; i < n; i++)
    {
      a[0][i] = a[0][i] + a[0][i-1];
    }
  for(int j = 1; j < n; j++)
    {
      a[j][0] = a[j][0] + a[j-1][0];
    }
```

```
for (int i=1; i<=n; i++)
  {
  for (int j=1; j<=m; j++)
    {
      if (s1[i-1] == s2[j-1])
        dp[i][j] = 1 + dp[i-1][j-1];
    }
  else
    dp[i][j] = max (dp[i-1][j], dp[i][j-1]);
  }
}
printf("%d\n", dp[n][m]);
}
```

Result:

Thus the program to find the longest substring between two string was written and executed successfully

Output	Expected
a a b a z b	2
A B C D A B C D	4



31

Longest Common Subsequence

Aim:

To find the length of longest common subsequence between two strings.

Program:

```
#include <stdio.h>
#include <string.h>
int max (int a, int b)
 {
  return (a>b)? a:b;
 }
int main ()
{
 char s1[100];
 char s2[100];
 scanf("%s", s1);
 scanf("%s", s2);
 int n = strlen(s1);
 int m = strlen(s2);
 int dp[n+1][m+1];
 for(int i=0; i<=n; i++)
  {dp[i][0]=0;
 }
 for (int j=0; j<=n; j++)
  {dp[0][j]=0;
 }
```

```
if (dp[i] > max)
    max = dp[i];
}
printf("%d", max);
}
```

Result:

Thus the program to find the longest non-decreasing subsequence has been executed successfully.

Input	Output
9 1 3452223 7 1224576	6 6

Longest non-decreasing Subsequence

Aim:

To find the length of longest Non-decreasing subsequence in a given sequence.

~~Algorithm~~

Program:

```
#include <stdio.h>
int main(){
  int n;
  scanf("%d",&n);
  int arr[n];
  int dp[n];
  for(int i=0;i<n;i++)
  {
   scanf("%d",&arr[i]);
   dp[i]=1;
  }
  int max=1;
  for(int i=1;i<n;i++){
    for(int j=0;j<i-1;j++){
      if(arr[j]<=arr[i]){
        if(dp[j]+1>dp[i]){
          dp[i]=dp[j]+1;
        }
      }
    }
  }
```

Input	Output
11	7
1 0 9 7 6 5 1 2 3 8 4 7	
5	4
1 2 3 4 4	
5	1
1 1 2 3 4	

Result:

Thus the program to find the duplicate element has been executed successfully.

Input	Output
11 10976512384 7	7
5 1 2 3 4 4	4
5 1 1 2 3 4	1

Competitive Programming
Finding Duplicates

Aim:
To find the duplicate element present in an list of elements.

Program:

```
#include <stdio.h>
int main()
{
  int n;
  scanf("%d",&n);
  int arr[n];
  for(int i=0;i<n;i++)
  {
    scanf("%d",&n);
  }
  for(int i=0;i<n;i++)
  {
    for(int j=0;j<n;j++)
    {
      if(arr[i]==arr[j] && i!=j)
        printf("%d",arr[i]);
    }
  }
}
```

Input	Output
11 10 9 7 6 5 1 2 3 8 4 7	7
5 1 2 3 44	4
5 1 1 2 3 4	1

2)

Finding Duplicate O(n) - Time

Aim:
To find the duplicate element in O(n) time.

Program:

```
#include <stdio.h>
int main()
{
 int n;
 scanf("%d",&n);
 int arr[n];
 for(int i=0; i<n; i++)
 {scanf("%d",&arr[i]);
 }
 for(int i=0;i<n;i++)
  {
  for(int j=0;j<n;j++)
   {
    if (arr[i]==arr[j] && i!=j)
     printf("%d", arr[i]);
   }
  }
}
```

Result:
Thus the program was written and executed successfully

Input	Output
11 10 9 7 6 5 1 2 3 8 4 7	7
5 1 2 3 44	4
5 1 1 2 3 4	1

Print Intersection of 2 Sorted Array

Aim:

To print the common elements in an sorted array in $O(m+n)$ time.

Program:

```
#include <stdio.h>
int main()
{ int t;
 scanf("%d",&t);
 while(t--)
 { int n1,n2;
 scanf("%d",&n1);
 int a[n1];
 for(int i=0;i<n1;i++)
   scanf("%d",&a[i]);
   int b[n2];
 for(int i=0;i<n2;i++)
   scanf("%d",&b[i]);
 int i=0,j=0;
 while(i<n1 && j<n2)
   if(a[i]==b[j]){
     printf("%d",a[i]);
    i++,j++;
 }
```

Input	Output
1 310157 6 2710155 7246	1057

```
else if (a[i] < b[j]
{
  i++;
}
else
  j++
}
printf("\n");
}
```

Result:

Thus the program has been written and executed successfully.

Pair with Difference: $O(n^2)$ - Time Complexity

Aim:

Given an array A of sorted integers another non-negative integer, find if there exists 2 numbers A such that A[j] - A[i] = i != j

Program:

```
#include <stdio.h>
int main()
{
 int n,k;
 scanf("%d",&n);
 int a[n];
 for(int i=0;i<n;i++)
 scanf("%d",&a[i]);
 scanf("%d",&k);
 int i=0; j=1;
 while(i<n && j<n){
   if(i==j)
     j++;
   }
 else if(a[j]-a[i]==k)
 {printf("1");
   return 0;
 }
```

Input	Output
3 1 3 5 4	1

```
else if (a[j]-a[i]<k)
    j++;
else
    i++
printf("0");
}
```

Result:

The program has been written and executed sucessfully.

Pair with Difference, O(n) - Time Complexity

Aim: To check if there exists 2 numbers in a sorted array of numbers where their difference is equal to target.

Program:

```
#include <stdio.h>
int main()
{
 int n, k;
 scanf("%d", &n);
 int a[n];
 for(int i=0;i<n;i++)
  scanf("%d", &a[i]);
 scanf("%d", &k);
 int i=0, j=1;
 while (i<n && j<n)
 { if (i==j)
    j++;
  else if (a[j]-a[i]==k)
     Printf("1");
     return 0;
  else if (a[j]-a[i]<k
    j++;
  else
    i++;
 }
```

Input	Output
3 1 3 5 4	1

```
    printf("0");
    return 0;
}
```

Result:

The program has been written and executed successfully.